

TRABALHO PRÁTICO FINAL

CURSO	UNIDADE CURRICULAR
DESENVOLVIMENTO DE SOFTWARE	PROGRAMAÇÃO SERVIDOR-WEB (PSW)
DOCENTE	
MÁRIO AMORIM	

PROJETO: APLICAÇÃO WEB COM API REST (TEMA À ESCOLHA)

OBJETIVOS

- Desenvolver uma aplicação web backend com Node.js e Express.
- Modelar e criar uma base de dados relacional em MySQL.
- Construir uma API REST completa (CRUD) sobre a base de dados.
- Aplicar relações entre tabelas com chaves estrangeiras e JOINS.
- Testar e documentar os endpoints da API.

DESCRIÇÃO

- Desenvolve uma aplicação web de gestão, com tema à escolha (ex.: ginásio, biblioteca, clínica, oficina, loja), composta por uma base de dados MySQL com, no mínimo, três tabelas relacionadas e uma API REST em Node.js + Express que permita listar, consultar, criar, atualizar e eliminar registos. Como valorização, pode ser desenvolvido um frontend simples em HTML que consuma a API. O frontend é opcional e feito em HTML, CSS e JavaScript.

REQUISITOS MÍNIMOS

Base de dados (MySQL)

- No mínimo, 3 tabelas relacionadas entre si.
- Pelo menos 1 relação de chave estrangeira (FK) entre tabelas.
- Dados de exemplo inseridos (mínimo de 3 registos por tabela).
- Ligação à base de dados com o módulo mysql2 (pool), num ficheiro db.js.
- Script dump.sql com a criação das tabelas e as inserções de exemplo.

2025/2026

API REST (Node.js + Express)

- GET /api/<entidade> – listar todos os registos.
- GET /api/<entidade>/:id – obter um registo específico.
- POST /api/<entidade> – criar um novo registo.
- PUT /api/<entidade>/:id – atualizar um registo existente.
- DELETE /api/<entidade>/:id – eliminar um registo.
- Respostas em JSON, com códigos HTTP adequados (200, 201, 400, 404, 500) e validação dos dados recebidos.
- Pelo menos uma rota com JOIN entre tabelas, devolvendo dados combinados (ex.: produto + nome da categoria).
- Pelo menos um endpoint de relação por chave estrangeira (ex.: GET /api/categorias/:id/produtos).

REQUISITOS ADICIONAIS (BÓNUS)

Frontend em HTML (valorização)

- Listar os registos das tabelas numa página HTML (via fetch).
- Adicionar, editar e eliminar dados através de formulários.
- Interligar entidades (ex.: associar um produto a uma categoria).

Boas práticas (valorização)

- Código organizado em ficheiros e pastas (server.js, db.js e public/, se houver frontend).
- Tratamento de erros e validação dos dados de entrada.
- Ficheiro README.md com instruções de instalação e execução.

REGRAS

O Trabalho Prático deve:

- Ser realizado em grupos de, preferencialmente, 2 elementos (máximo 3 elementos ou individual);
- Ser submetido via classroom na data definida e ser apresentado, presencialmente, na aula respetiva. O aluno que não compareça à apresentação, sem justificação válida por regulamento, não terá avaliação nesta componente;
- Ser entregue em formato .ZIP (sem a pasta node_modules), com o nome PSW-TP-NOME-NOME (em que NOME corresponde ao nome dos alunos). Basta o envio de um trabalho por grupo.
 - Alternativa! Caso o classroom não permita, pode enviar o link wetransfer para: mario.amorim@my.istec.pt
- Código completo do projeto: server.js, db.js e, se existir, a pasta public/.
- Ficheiro dump.sql com a criação das tabelas e as inserções de exemplo.
- Exportação do Insomnia / Postman / Thunder Client com todos os pedidos testados.
- Ficheiro README.md com instruções de utilização.

APRESENTAÇÃO

A apresentação pode ser preparada à posteriori e não precisa de ser entregue no classroom. Esta apresentação deve ser fluída, organizada e num formato de **pitch de 10 minutos (máximo)**. No final, é reservado tempo de debate com o “cliente”.

Para preparar a apresentação final pode organizar o trabalho num *PPT* ou outra ferramenta a gosto. Deve incluir:

- *Slide* capa;
- *Slide* de contextualização do projeto com requisitos;
- *Slides* com o trabalho desenvolvido – pode incluir prints, vídeos ou outros elementos, ou apresentação prática do projeto;
- *Slide* com mapa da gestão de tarefas por elemento da equipa responsável e respetiva percentagem de conclusão;
- *Slide* com conclusão sobre o desempenho/a evolução do trabalho;
- *Slide* com autoavaliação da equipa e/ou individualmente por cada membro (se achar pertinente) relativamente.

PARÂMETROS DE AVALIAÇÃO

- API REST e Base de Dados 60%
 - Funcionalidade da API (CRUD completo)
 - Boas práticas e validação de código
 - Estrutura da base de dados e relações (FK, JOIN)
 - Testes à API (Insomnia / Postman / Thunder Client)
 - Frontend em HTML (valorização)
 - Cumprimento de prazos
- Apresentação 40%
 - Originalidade e Criatividade
 - Organização e estrutura
 - Qualidade visual
 - Clareza e objetividade na expressão oral
 - Trabalho em grupo
 - Defesa

Bom trabalho! ☺